M10 Learner Book

Docker + Airflow - full beginner teaching edition

Release Candidate 2 - beginner-sufficiency content repair

How to use this book

Read this book before treating Docker or Airflow as a list of commands. Docker gives you a reproducible runtime boundary; Airflow gives you an operational workflow boundary. The important skill is knowing which problem belongs to which layer.

- Book = explanation and mental model.
- Guided labs = follow-with-me execution.
- Independent tasks = transfer without step-by-step commands.
- Module Practical = fresh Docker + Airflow build.
- A real Docker/Airflow runtime smoke test remains mandatory before the course-wide FINAL release.

Whole-module mental model

Docker does not make incorrect Python correct. Airflow does not make a non-idempotent load safe. M10 assumes the reliability principles from M08 and the transformation discipline from M09 already exist.

M10-L01 - Why Containers Exist

By the end of this lesson, you should be able to explain why containers solve reproducibility problems that virtual environments alone do not fully solve.

Plain-English explanation

A Python virtual environment controls Python packages. It does not fully control the operating-system libraries, system tools, filesystem layout, user identity or service dependencies around your program. Docker packages an application into an image definition so another compatible Docker host can create the same runtime boundary.

Key terms

Term	Meaning
image	built artifact created from a Dockerfile/build context
container	running instance of an image
host	the machine/OS running Docker
build context	files Docker may send to the image build

Walk through the example

> ***Machine A: Python 3.13 + libpq + app requirements***
> ***Machine B: Python 3.12 + missing libpq***
> ***Docker image: declared Python base + declared dependencies***
> ***Both machines run the same image definition.***

How to reason about it

- Ask what belongs in the image versus runtime configuration.
- Ask what must survive container replacement.
- Ask whether communication is host-to-container or container-to-container.
- Ask whether sensitive values are present in source-controlled files.

Common mistakes

- A container does not fix bad business logic or invalid source data.
- A container is not a place to make undocumented manual fixes. Rebuild the image instead.
- The image normally contains code/runtime; durable production data should live outside the container writable layer.

Debug checklist

1. Confirm you are in the intended project directory.

2. Validate configuration (`docker compose config`) before starting.

3. Inspect service/container state.

4. Inspect logs for the specific service.

5. Check mounts, ports, service hostname and secret path before rewriting application code.

M10-L02 - Images and Containers

By the end of this lesson, you should be able to distinguish build-time image state from runtime container state and inspect the lifecycle safely.

Plain-English explanation

`docker build` executes Dockerfile instructions and produces an image. `docker run` creates a container from that image. Multiple containers can be created from one image. Removing a container does not automatically remove the image.

Key terms

Term	Meaning
build	create an image
run	create/start a container
writable layer	runtime changes attached to one container
--rm	remove the container automatically when it exits

Walk through the example

```
docker build -t m10-pipeline:dev app
docker image ls
docker run --rm m10-pipeline:dev
docker ps -a
```

How to reason about it

- Ask what belongs in the image versus runtime configuration.
- Ask what must survive container replacement.
- Ask whether communication is host-to-container or container-to-container.
- Ask whether sensitive values are present in source-controlled files.

Common mistakes

- Editing a running container and assuming the change is reproducible.
- Confusing stopped container with deleted image.
- Saving important database state only in a disposable container layer.

Debug checklist

1. Confirm you are in the intended project directory.

2. Validate configuration (`docker compose config`) before starting.

3. Inspect service/container state.

4. Inspect logs for the specific service.

5. Check mounts, ports, service hostname and secret path before rewriting application code.

M10-L03 - Writing a Dockerfile

By the end of this lesson, you should be able to build a readable Python data image with declared dependencies, a non-root user and a predictable command.

Plain-English explanation

A Dockerfile is executable build documentation. Its order matters because Docker can reuse unchanged build layers. Copying dependency metadata before application source often lets dependency installation stay cached when only source code changes.

Key terms

Term	Meaning
FROM	base image
WORKDIR	default directory for later instructions/runtime
COPY	put build-context files into the image
RUN	execute a build-time command
USER	choose runtime identity
CMD	default command when container starts

Walk through the example

```
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
COPY src ./src
RUN useradd --create-home appuser
USER appuser
CMD ["python", "src/pipeline.py"]
```

How to reason about it

- Ask what belongs in the image versus runtime configuration.
- Ask what must survive container replacement.
- Ask whether communication is host-to-container or container-to-container.
- Ask whether sensitive values are present in source-controlled files.

Common mistakes

- Copying `.env`, secrets or huge datasets into the build context.
- Running as root without need.
- Using `latest` everywhere and losing version predictability.
- Putting commands you perform manually after startup instead of in the build/configuration.

Debug checklist

1. Confirm you are in the intended project directory.

2. Validate configuration (`docker compose config`) before starting.

3. Inspect service/container state.

4. Inspect logs for the specific service.

5. Check mounts, ports, service hostname and secret path before rewriting application code.

M10-L04 - Ports

By the end of this lesson, you should be able to map host access to container ports and diagnose “the service runs but I cannot reach it.”

Plain-English explanation

Containers have their own network namespace. A program listening on port 8080 inside a container is not automatically reachable on host port 8080. Port publishing creates an explicit host-to-container mapping.

Key terms

Term	Meaning
container port	port the process listens on inside the container
host port	port exposed on the host
publish	create host-to-container mapping
EXPOSE	documentation/metadata; not host publication by itself

Walk through the example

```
ports:
  - "18080:8080"
# browser on host -> http://localhost:18080
# application listens inside container on 8080
```

How to reason about it

- Ask what belongs in the image versus runtime configuration.
- Ask what must survive container replacement.
- Ask whether communication is host-to-container or container-to-container.
- Ask whether sensitive values are present in source-controlled files.

Common mistakes

- Swapping host and container sides.
- Changing the application port when only the host port conflicts.
- Assuming EXPOSE alone opens a host port.

Debug checklist

1. Confirm you are in the intended project directory.

2. Validate configuration (`docker compose config`) before starting.

3. Inspect service/container state.

4. Inspect logs for the specific service.

5. Check mounts, ports, service hostname and secret path before rewriting application code.

M10-L05 - Volumes and Persistence

By the end of this lesson, you should be able to choose named volumes and bind mounts based on what must survive and who must see the files.

Plain-English explanation

A container should be replaceable. Database data often must survive that replacement, so it belongs in a persistent volume. During learning, source CSVs and generated reports are easier to inspect through bind mounts that map a host folder into the container.

Key terms

Term	Meaning
named volume	Docker-managed persistent storage
bind mount	explicit host path mapped into container
container layer	disposable runtime filesystem changes
persistence	state survives container replacement

Walk through the example

```
services:
  db:
    volumes:
      - db_data:/var/lib/postgresql/data
  pipeline:
    volumes:
      - ./data:/app/data
volumes:
  db_data:
```

How to reason about it

- Ask what belongs in the image versus runtime configuration.
- Ask what must survive container replacement.
- Ask whether communication is host-to-container or container-to-container.
- Ask whether sensitive values are present in source-controlled files.

Common mistakes

- Using `docker compose down -v` and being surprised that database data disappears.
- Keeping a database only in the container layer.
- Mounting over a directory that already contains important image files and hiding them.

Debug checklist

1. Confirm you are in the intended project directory.

2. Validate configuration (`docker compose config`) before starting.

3. Inspect service/container state.

4. Inspect logs for the specific service.

5. Check mounts, ports, service hostname and secret path before rewriting application code.

M10-L06 - Networks and Service Communication

By the end of this lesson, you should be able to connect Compose services by service name and distinguish host networking from container networking.

Plain-English explanation

Inside the `pipeline` container, `localhost` means the pipeline container itself. The PostgreSQL server is in another container. Compose provides a network where the database service can be reached by its service name such as `db`.

Key terms

Term	Meaning
service name	DNS-style hostname on the Compose network
localhost	the current network namespace
container-to-container	traffic between services on the Compose network
host-to-container	traffic entering via a published host port

Walk through the example

```
services:
  db:
    image: postgres:17-alpine
  pipeline:
    environment:
      DB_HOST: db
      DB_PORT: "5432"
```

How to reason about it

- Ask what belongs in the image versus runtime configuration.
- Ask what must survive container replacement.
- Ask whether communication is host-to-container or container-to-container.
- Ask whether sensitive values are present in source-controlled files.

Common mistakes

- Using DB_HOST=localhost inside the pipeline service.
- Publishing the database port to the host when only internal services need it.
- Debugging PostgreSQL itself before checking the hostname/network.

Debug checklist

1. Confirm you are in the intended project directory.

2. Validate configuration (`docker compose config`) before starting.

3. Inspect service/container state.

4. Inspect logs for the specific service.

5. Check mounts, ports, service hostname and secret path before rewriting application code.

M10-L07 - Compose, Configuration and Secrets

By the end of this lesson, you should be able to define a multi-service stack while keeping ordinary configuration separate from credentials.

Plain-English explanation

Compose describes services, networks, volumes, health checks and configuration as code. Ordinary non-sensitive values can be environment variables. Passwords/tokens should not be embedded directly in source-controlled YAML or ordinary environment blocks; use an appropriate secret mechanism.

Key terms

Term	Meaning
service	one runnable component definition
depends_on	startup/dependency relationship
healthcheck	probe used to determine readiness/health
secret	sensitive value supplied separately from ordinary config

Walk through the example

```
secrets:
  db_password:
    file: ./secrets/db_password.txt
services:
  db:
    secrets: [db_password]
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
```

How to reason about it

- Ask what belongs in the image versus runtime configuration.
- Ask what must survive container replacement.
- Ask whether communication is host-to-container or container-to-container.
- Ask whether sensitive values are present in source-controlled files.

Common mistakes

- Committing a real password because “it is only a local Compose file.”
- Assuming deleting the line later removes the secret from Git history.
- Using `depends_on` alone as proof that the database is ready to accept connections.

Debug checklist

1. Confirm you are in the intended project directory.

2. Validate configuration (`docker compose config`) before starting.

3. Inspect service/container state.

4. Inspect logs for the specific service.

5. Check mounts, ports, service hostname and secret path before rewriting application code.

M10-L08 - Why Orchestration Exists

By the end of this lesson, you should be able to explain what orchestration adds after the underlying pipeline code is already correct.

Plain-English explanation

A recurring data pipeline needs more than a script: it needs a schedule, dependencies, retry boundaries, run history, logs, operational states and historical reprocessing. Airflow coordinates those pieces. The business transformation should remain in normal code/dbt/Spark rather than being hidden inside a giant Dag file.

Key terms

Term	Meaning
orchestration	coordinate when/how work runs
Dag	workflow definition
task	operational unit of work
run	one logical execution of the workflow

Walk through the example

```
extract -> validate -> transform -> load
          | failure
          v
     downstream blocked
```

How to reason about it

- Ask what the task boundary represents operationally.
- Ask whether the task is safe to retry.
- Ask which logical data interval the run should process.
- Ask what evidence you would inspect after failure.

Common mistakes

- Turning every Python function into a separate Airflow task.
- Putting the entire data platform inside one unobservable task.
- Expecting Airflow retries to fix duplicate-producing writes.

Debug checklist

1. Identify the Dag run and exact failed task instance.

2. Read the task log before clearing/rerunning.

3. Classify the failure as transient or deterministic.

4. Check that retry/backfill will not duplicate durable data.

5. Repair the smallest responsible layer and rerun only what is safe.

M10-L09 - Airflow Architecture and UI

By the end of this lesson, you should be able to identify the Dag, Dag run, task instance and logs needed to diagnose an operational problem.

Plain-English explanation

A Dag defines the workflow. The scheduler decides which task instances are eligible to run according to schedule and dependencies. A task instance is a particular task in a particular Dag run. The UI exposes parsed Dags, runs, states and logs.

Key terms

Term	Meaning
Dag	workflow definition
Dag run	one workflow execution
task instance	one task in one run
scheduler	creates/schedules runnable task instances
log	execution evidence for one task attempt

Walk through the example

> ***Dag: daily_orders_training***
> ***Run: scheduled/manual instance for a logical interval***
> ***Task instance: validate in that run***
> ***Log: validation output/error for that attempt***

How to reason about it

- Ask what the task boundary represents operationally.
- Ask whether the task is safe to retry.
- Ask which logical data interval the run should process.
- Ask what evidence you would inspect after failure.

Common mistakes

- Treating “Dag is visible” as proof the run succeeded.
- Clearing a failed task before saving the error evidence.
- Looking only at the final task instead of tracing upstream state.

Debug checklist

1. Identify the Dag run and exact failed task instance.

2. Read the task log before clearing/rerunning.

3. Classify the failure as transient or deterministic.

4. Check that retry/backfill will not duplicate durable data.

5. Repair the smallest responsible layer and rerun only what is safe.

M10-L10 - Dags, Tasks and Dependencies

By the end of this lesson, you should be able to build a clear task graph where downstream work cannot run before prerequisites succeed.

Plain-English explanation

Task boundaries should represent operationally meaningful work with useful retry/logging boundaries. In TaskFlow, passing one task output into another creates dependencies. Dependencies can also be expressed explicitly when no data value must move.

Key terms

Term	Meaning
upstream	task that must complete before another
downstream	task waiting on an upstream dependency
dependency	ordering/eligibility relationship
task graph	the Dag structure

Walk through the example

```
from airflow.sdk import dag, task

@dag(schedule="@daily", start_date=..., catchup=False)
def pipeline():
  @task
  def extract(): return {"rows": 10}
  @task
  def validate(meta): return meta
  @task
  def load(meta): print(meta)
  load(validate(extract()))
```

How to reason about it

- Ask what the task boundary represents operationally.
- Ask whether the task is safe to retry.
- Ask which logical data interval the run should process.
- Ask what evidence you would inspect after failure.

Common mistakes

- Calling a decorated task as if it were ordinary Python outside the Dag definition.
- Creating dependencies that allow load before validation.
- Making task boundaries so tiny that the graph becomes noise.

Debug checklist

1. Identify the Dag run and exact failed task instance.

2. Read the task log before clearing/rerunning.

3. Classify the failure as transient or deterministic.

4. Check that retry/backfill will not duplicate durable data.

5. Repair the smallest responsible layer and rerun only what is safe.

M10-L11 - TaskFlow, Operators and Task Communication

By the end of this lesson, you should be able to choose task boundaries and pass small metadata rather than bulk datasets through orchestration metadata channels.

Plain-English explanation

TaskFlow is convenient for Python-first tasks. Operators/providers integrate external systems. Airflow task communication mechanisms are best for small metadata such as a file path, run identifier or row count—not for shipping a 5 GB DataFrame between tasks.

Key terms

Term	Meaning
TaskFlow	Python-decorator task style
operator	task abstraction often integrating an external system
XCom	small task-to-task metadata mechanism
durable data	files/tables/object storage outside XCom

Walk through the example

```
extract -> {"raw_path": "/data/orders.csv", "rows": 1000}
validate(metadata) -> {"valid_path": "/data/valid.parquet", "rows": 993}
```

How to reason about it

- Ask what the task boundary represents operationally.
- Ask whether the task is safe to retry.
- Ask which logical data interval the run should process.
- Ask what evidence you would inspect after failure.

Common mistakes

- Returning huge datasets from tasks.
- Passing passwords in task return values/logs.
- Creating tasks for pure helper functions that do not need independent operational state.

Debug checklist

1. Identify the Dag run and exact failed task instance.

2. Read the task log before clearing/rerunning.

3. Classify the failure as transient or deterministic.

4. Check that retry/backfill will not duplicate durable data.

5. Repair the smallest responsible layer and rerun only what is safe.

M10-L12 - Scheduling and Data Intervals

By the end of this lesson, you should be able to distinguish the time a run starts from the logical slice of data that run should process.

Plain-English explanation

A daily pipeline may start shortly after midnight but process the previous completed day. Reliable backfills/retries depend on deterministic interval-aware logic, not “whatever datetime.now() says when this task happens to run.”

Key terms

Term	Meaning
schedule	rule that creates runs
logical/data interval	the intended data window for a run
wall clock	actual current time
deterministic	same logical run selects same intended data

Walk through the example

> ***Run starts: 2026-08-28 00:05***
> ***Intended data: 2026-08-27 00:00 -> 2026-08-28 00:00***
> ***Task queries that interval explicitly.***

How to reason about it

- Ask what the task boundary represents operationally.
- Ask whether the task is safe to retry.
- Ask which logical data interval the run should process.
- Ask what evidence you would inspect after failure.

Common mistakes

- Using today()/now() inside every task and making historical runs process current data.
- Confusing the trigger timestamp with the business-date slice.
- Hard-coding one date into a Dag and then forgetting to remove it.

Debug checklist

1. Identify the Dag run and exact failed task instance.

2. Read the task log before clearing/rerunning.

3. Classify the failure as transient or deterministic.

4. Check that retry/backfill will not duplicate durable data.

5. Repair the smallest responsible layer and rerun only what is safe.

M10-L13 - Retries, Timeouts and Failure Behavior

By the end of this lesson, you should be able to configure bounded retries for transient failures and avoid retrying deterministic errors blindly.

Plain-English explanation

Retries are useful for temporary network/API/service failures. They are harmful when the task is non-idempotent or the failure will never change without human/configuration repair. Timeouts prevent a stuck task from consuming resources forever.

Key terms

Term	Meaning
retry	another task attempt after failure
retry delay	wait between attempts
execution timeout	maximum allowed task duration
transient failure	may succeed later
deterministic failure	same input/config likely fails again

Walk through the example

```
@task(retries=2, retry_delay=timedelta(minutes=1),
    execution_timeout=timedelta(minutes=10))
def extract(): ...
```

How to reason about it

- Ask what the task boundary represents operationally.
- Ask whether the task is safe to retry.
- Ask which logical data interval the run should process.
- Ask what evidence you would inspect after failure.

Common mistakes

- Retrying invalid credentials 20 times.
- Retrying a plain INSERT task that duplicates rows every attempt.
- Using unlimited/very large retries to hide a broken dependency.

Debug checklist

1. Identify the Dag run and exact failed task instance.

2. Read the task log before clearing/rerunning.

3. Classify the failure as transient or deterministic.

4. Check that retry/backfill will not duplicate durable data.

5. Repair the smallest responsible layer and rerun only what is safe.

M10-L14 - Parameters and Task Communication

By the end of this lesson, you should be able to pass run-specific configuration explicitly and keep orchestration metadata small and inspectable.

Plain-English explanation

Tasks often need a business date, partition, source path or run identifier. Passing those values explicitly makes the workflow reproducible. Hidden global variables and wall-clock calls make historical runs harder to trust.

Key terms

Term	Meaning
parameter	input supplied to workflow/task
run config	values specific to one execution
metadata	small descriptive data
bulk data	large records/files that should live in durable storage

Walk through the example

extract -> {"raw_path":"/data/raw/2026-08-27.csv", "row_count":1000}
validate(meta) -> {"valid_path":"/data/valid/2026-08-27.parquet", "rows":998}

How to reason about it

- Ask what the task boundary represents operationally.
- Ask whether the task is safe to retry.
- Ask which logical data interval the run should process.
- Ask what evidence you would inspect after failure.

Common mistakes

- Passing a DataFrame through XCom.
- Logging tokens/passwords inside metadata.
- Using a global “current_date” variable that changes during backfill.

Debug checklist

1. Identify the Dag run and exact failed task instance.

2. Read the task log before clearing/rerunning.

3. Classify the failure as transient or deterministic.

4. Check that retry/backfill will not duplicate durable data.

5. Repair the smallest responsible layer and rerun only what is safe.

M10-L15 - Connections, Variables and Secrets

By the end of this lesson, you should be able to keep connection credentials outside Dag source and know which configuration belongs in Airflow metadata/configuration systems.

Plain-English explanation

A Dag committed to Git should not reveal database passwords or API tokens. Airflow connection/secret-backend mechanisms hold connection material; variables can hold non-secret runtime configuration, but should not become a dumping ground for credentials.

Key terms

Term	Meaning
connection	structured external-system connection info
variable	runtime configuration value
secret backend	external/specialized credential retrieval mechanism
least exposure	only the task/system that needs the secret should receive it

Walk through the example

```
# conceptual
conn_id = "warehouse"
# task/provider retrieves the connection at runtime
# password is not written in the Dag file
```

How to reason about it

- Ask what the task boundary represents operationally.
- Ask whether the task is safe to retry.
- Ask which logical data interval the run should process.
- Ask what evidence you would inspect after failure.

Common mistakes

- Hard-coding passwords in Python.
- Printing connection objects/tokens in task logs.
- Putting every setting into Airflow Variables instead of version-controlled configuration where appropriate.

Debug checklist

1. Identify the Dag run and exact failed task instance.

2. Read the task log before clearing/rerunning.

3. Classify the failure as transient or deterministic.

4. Check that retry/backfill will not duplicate durable data.

5. Repair the smallest responsible layer and rerun only what is safe.

M10-L16 - Sensors and Waiting

By the end of this lesson, you should be able to model waiting for external conditions without writing an endless custom polling loop.

Plain-English explanation

Some workflows should wait for a file, partition or external condition. A sensor represents that waiting as orchestrated state with defined polling/timeout behavior. The design should still include a failure/timeout path rather than “wait forever.”

Key terms

Term	Meaning
sensor	task that waits for a condition
poke/poll	check condition repeatedly
timeout	maximum wait
event-driven alternative	start/continue work from an external event when supported

Walk through the example

```
wait_for_input -> extract -> validate
# if input never arrives by timeout: fail/alert rather than hang forever
```

How to reason about it

- Ask what the task boundary represents operationally.
- Ask whether the task is safe to retry.
- Ask which logical data interval the run should process.
- Ask what evidence you would inspect after failure.

Common mistakes

- Using `while True: sleep()` inside a normal task for hours.
- Waiting without a timeout or alert path.
- Using a sensor when the upstream system can trigger the workflow directly.

Debug checklist

1. Identify the Dag run and exact failed task instance.

2. Read the task log before clearing/rerunning.

3. Classify the failure as transient or deterministic.

4. Check that retry/backfill will not duplicate durable data.

5. Repair the smallest responsible layer and rerun only what is safe.

M10-L17 - Logs and Debugging

By the end of this lesson, you should be able to diagnose a failed task from orchestration state down to the underlying code/data cause.

Plain-English explanation

Airflow tells you which run/task/attempt failed and gives you logs. It does not replace debugging the actual script, SQL, API call or data contract. A disciplined investigation moves from orchestration evidence into the underlying layer.

Key terms

Term	Meaning
state	queued/running/success/failed/retry etc.
attempt	one execution try for a task instance
task log	stdout/stderr/framework logging for that attempt
root cause	underlying code/data/config problem

Walk through the example

> ***Dag page -> failed Dag run -> failed task instance -> task log***
> ***Required column customer_id missing***
> ***=> schema/data-contract problem; do not retry blindly***

How to reason about it

- Ask what the task boundary represents operationally.
- Ask whether the task is safe to retry.
- Ask which logical data interval the run should process.
- Ask what evidence you would inspect after failure.

Common mistakes

- Clearing/rerunning before saving the error.
- Changing unrelated tasks during incident repair.
- Treating every red task as an Airflow bug.

Debug checklist

1. Identify the Dag run and exact failed task instance.

2. Read the task log before clearing/rerunning.

3. Classify the failure as transient or deterministic.

4. Check that retry/backfill will not duplicate durable data.

5. Repair the smallest responsible layer and rerun only what is safe.

M10-L18 - Catchup, Backfills and Reprocessing

By the end of this lesson, you should be able to reprocess historical intervals without corrupting already-correct target state.

Plain-English explanation

Catchup controls whether scheduled historical runs are created automatically from a start date. A backfill explicitly creates runs for historical dates/intervals. Airflow can create those runs, but the underlying tasks still need interval-aware, idempotent behavior.

Key terms

Term	Meaning
catchup	automatic historical scheduled runs
backfill	explicit historical execution request
reprocessing	run a past logical interval again
idempotency	repeat without corrupting durable result

Walk through the example

```
airflow backfill create --dag-id daily_orders_training \
  --start-date 2026-08-17 --end-date 2026-08-19
# verify exact CLI/help for installed Airflow release
```

How to reason about it

- Ask what the task boundary represents operationally.
- Ask whether the task is safe to retry.
- Ask which logical data interval the run should process.
- Ask what evidence you would inspect after failure.

Common mistakes

- Assuming Airflow prevents duplicates for you.
- Using current date in tasks so backfill processes today.
- Deleting good historical data before the replacement run is validated.

Debug checklist

1. Identify the Dag run and exact failed task instance.

2. Read the task log before clearing/rerunning.

3. Classify the failure as transient or deterministic.

4. Check that retry/backfill will not duplicate durable data.

5. Repair the smallest responsible layer and rerun only what is safe.

M10-L19 - Integration and Production-Style Orchestration

By the end of this lesson, you should be able to separate container configuration, orchestration and business pipeline logic in a reproducible local platform.

Plain-English explanation

A maintainable local project lets another engineer start services from documented files, see the Dag, understand task boundaries, rerun safely and inspect logs. Docker/Compose define runtime services; Airflow defines workflow coordination; Python/dbt code defines data behavior.

Key terms

Term	Meaning
Dockerfile	image definition
Compose	multi-service runtime configuration
Dag	workflow definition
pipeline code	business/data logic
evidence	logs, audit files, task states, durable outputs

Walk through the example

Docker/Compose: app + db + volume + secret
Airflow Dag: extract -> validate -> transform -> load
Pipeline code: validation/transformation/reconciliation
Evidence: logs + audit + rerun/backfill result

How to reason about it

- Ask what the task boundary represents operationally.
- Ask whether the task is safe to retry.
- Ask which logical data interval the run should process.
- Ask what evidence you would inspect after failure.

Common mistakes

- Putting all components in one Dockerfile.
- Putting all business transformation directly inside the Dag file.
- A successful green Dag with no reconciliation/audit evidence.

Debug checklist

1. Identify the Dag run and exact failed task instance.

2. Read the task log before clearing/rerunning.

3. Classify the failure as transient or deterministic.

4. Check that retry/backfill will not duplicate durable data.

5. Repair the smallest responsible layer and rerun only what is safe.